

rag-lab 系统架构说明

中文技术语料上的 RAG 分块策略评测系统 —— 3 种策略 × 3 种粒度 × 90 题

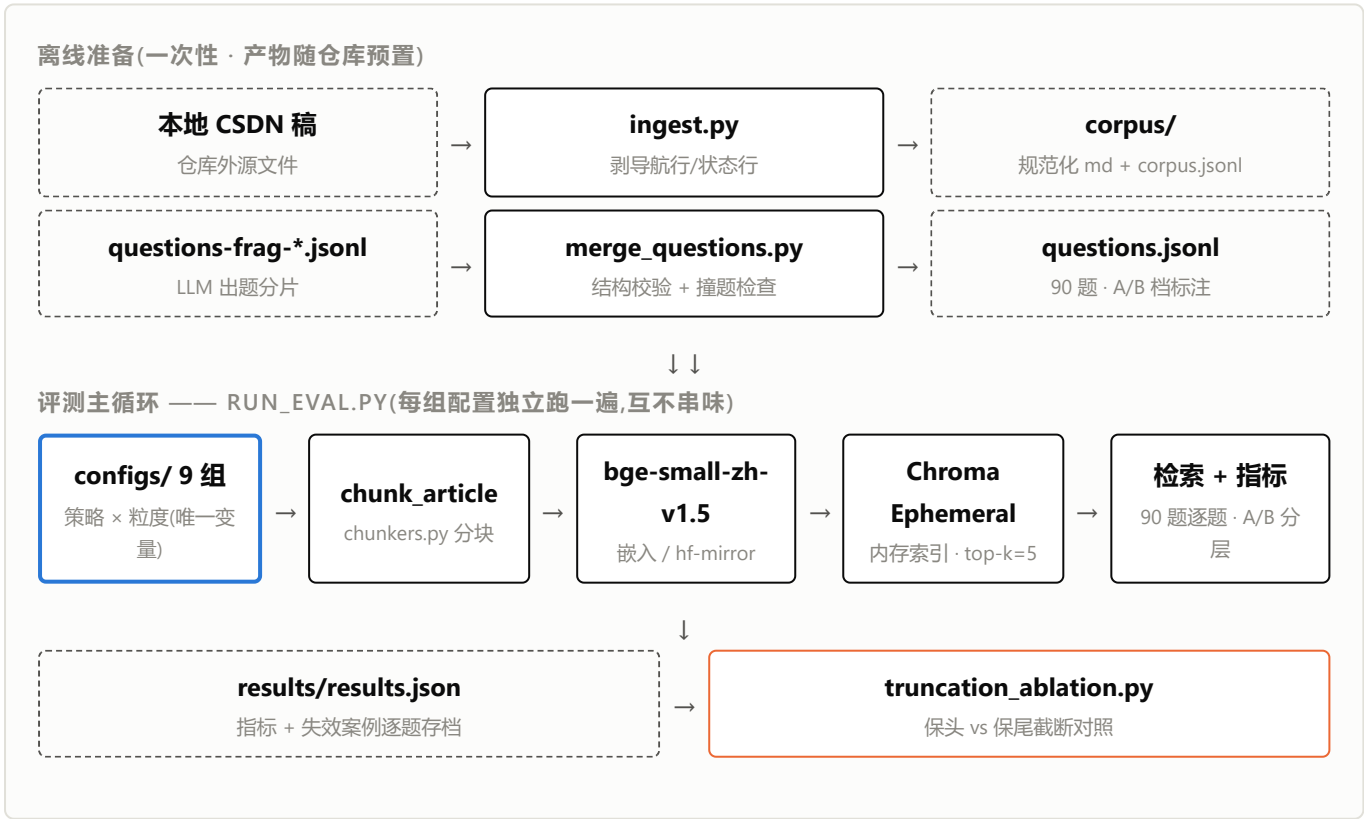
v1.0 · 2026-09-07 · 仓库:github.com/ethanliang2016/rag-lab · 配套文章:《RAG 分块策略实测:固定窗口 vs 递归切分 vs 结构感知,用 90 道题说话》

1 系统概述

rag-lab 是一套单机、纯本地运行的 RAG 检索层评测系统。它把 9 篇中文技术长文(已发布 6 篇、未发稿 3 篇)按 9 组"分块策略 × 粒度"配置切块、嵌入、入内存向量库,再用 90 道标注题逐组检索,输出可横向对比的命中率指标(Hit@1 / Hit@5 / MRR@5,按 A/B 档分层)。系统的唯一目的是回答一个问题:分块策略的差异,在数字上到底有多大——因此一切设计都服从"单变量对照"这一原则。



2 架构总览



图例:□ 处理组件 · □ 数据产物 · □ 实验变量入口 · □ 旁路对照实验。蓝框是全系统唯一自变量,其余环节全部固定:嵌入模型 bge-small-zh-v1.5、向量库 Chroma、top-k=5、overlap=50 字符。

3 组件说明

组件	职责	输入 → 输出
src/ingest.py	语料清洗入库:按 MANIFEST 逐篇读取本地稿,剥系列导航行与状态注记行,代码块/表格原样保留	本地 md → corpus/*.md + corpus.jsonl
src/merge_questions.py	题库分片合并:结构校验 (article_id/tier/id 合法性)、元指涉检查、6 字滑窗跨篇撞题初筛	questions-frag-*.jsonl → questions.jsonl
src/chunkers.py	三种分块策略实现:固定字符窗口 / LangChain 递归切分(中文分隔符)/ Markdown 结构感知(按标题分节、代码块不切断);含代码块切断统计	文章 md → chunk 列表
src/run_eval.py	主评测循环:分块 → 嵌入 → 建 Chroma 内存索引 → 90 题逐题检索 → 指标计算与附加观察(切断数/跨域干扰/失效案例)	corpus + questions + configs → results/results.json
src/truncation_ablation.py	截断伪影对照:fixed-1024 的块分别保头/保尾 510 token 送嵌入,隔离"截断方向 × 信号位置"效应	corpus + questions → 截断对照数据

4 数据结构

corpus.jsonl(语料元数据,每行一篇)

```
{
  "id": "dsh-startup",
  "domain": "ai",
  "title": "DeepSeek Harness 上手实录",
  "chars": 8433,
  "source": "AI情报收集/深度实测/DSH系列/...md"
}
```

语料 slug,同目录 md 文件名
ai | java(老文入库后启用)
清洗后字符数
仓库外源路径

questions.jsonl(测试集,每行一题)

```
{
  "id": "q-dsh-01",
  "question": "DSH 用 npm 发了多少个版本?",
  "article_id": "dsh-startup",
  "tier": "B"
}
```

ground truth(文章级)
A=主题级 27 题 / B=细节级 63 题

results/results.json(每组配置一条记录)

```
{
  "config": {
    "strategy": "fixed",
    "size": 1024,
    "chunks": 76,
    "codeblock_cuts": 7,
    "cross_domain": 0
  },
  "results": {
    "total_chunks": 76,
    "codeblock_cuts": 7,
    "cross_domain": 0
  }
}
```

该配置切出的总块数
代码块被切断次数
java 文混入 ai 题 top-5 次数

```
"metrics": {"overall": {...}, "A": {...}, "B": {...}},  
            # 每档含 n / hit1 / hit5 / mrr5  
"misses": [ ... ],      # Hit@1 未中的题逐题存档(写作素材)  
"seconds": 11.8}
```

5 关键设计决策

决策	理由
Chroma 用 EphemeralClient(内存)	9 组配置各自分块、各自嵌入,持久化省不了嵌入计算,只省检索层(可忽略);跑完即弃,无旧向量残留污染对照。切持久化的时点在重排序实验(固定 chunk 只换 reranker)
变量收敛到 configs 一处	策略 × 粒度是唯一自变量,嵌入模型/向量库/top-k/overlap 全程固定——保证 9 组数字可横向比较
ground truth 标文章级	chunk 级人工成本过高;口径宽松(top-k 命中目标文章即记 hit),数字只用于组间对比,不代表端到端问答质量
模型走 hf-mirror 下载	国内网络直连 huggingface.co 不可达,脚本内 <code>HF_ENDPOINT</code> 自动指镜像,clone 即跑
失效案例逐题存档	misses 字段留全部 Hit@1 未中题——评测的残差本身是下一篇(重排序)的选题来源
ingest.py 不进复现主链	其 MANIFEST 指向本机仓库外路径,对外预期 [miss];语料已随仓库预置,clone 后直接 run_eval

6 运行步骤

环境要求

Python 3.10+;CPU 即可(每组 9~14 秒);无需 GPU、无需联网账号(嵌入模型自动从 hf-mirror 下载,首次运行约 90MB)。

```
git clone https://github.com/ethanliang2016/rag-lab && cd rag-lab
pip install -r requirements.txt
python src/run_eval.py                # 全部 9 组(CPU 约 2 分钟)
python src/run_eval.py --config configs/recursive-512.json  # 单组
python src/truncation_ablation.py     # 截断伪影对照
```

依赖:sentence-transformers ≥2.7 / chromadb ≥0.5 / langchain-text-splitters ≥0.2。结果输出至 `results/results.json`。

7 目录结构

```
rag-lab/
├─ corpus/                # 语料:规范化 md + corpus.jsonl 元数据
├─ questions.jsonl        # 测试集(90 题,A/B 档标注)
├─ questions-frag-*.jsonl # 出题分片(工作留档,复现测试集构建)
├─ configs/               # 9 组实验配置({strategy, size})
├─ src/
│   ├─ ingest.py          # 语料清洗入库(本地再生成用)
│   ├─ merge_questions.py # 题库合并 + 校验 + 撞题检查
│   ├─ chunkers.py        # 三种分块策略 + 切断统计
│   └─ run_eval.py        # 主评测:分块→嵌入→检索→指标
```

```
|   └─ truncation_ablation.py  # 截断伪影对照(保头 vs 保尾)
|─ results/results.json      # 跑批结果 + 失效案例逐题存档
|─ requirements.txt
|─ README.md                # 含 Mermaid 管线图与复现指引
```

8 指标口径

- **主指标:** Hit@1 / Hit@5 / MRR@5, ground truth 在文章级——top-k 命中目标文章即记 hit, 按 A 档(主题级)/ B 档(细节级)分层报数;
- **附加观察:** 代码块被切断次数、跨域干扰计数(java 文混入 ai 题 top-5)、失效案例存档;
- **诚实口径:** n=90 报分层观察, 不做显著性检验; 全实验 810 次检索为单次运行; 结论不外推英文语料与其他文体。